

De la interpolación clásica a la Super-Resolución con Deep Learning y Transformers

Documentación complementaria al notebook [Zoom_Resize_Image.ipynb](#), pensada para una **audiencia mixta**: cada sección empieza con una explicación en lenguaje llano y una analogía (💬 / 🎯) que puedes usar tal cual en la presentación, y después mantiene el detalle técnico y las fórmulas para quien quiera profundizar.

El notebook cubre los fundamentos (píxel, PPI/DPI) y la interpolación clásica (Nearest, Bilineal, Bicúbica, Lanczos). Este documento explica **por qué** esos métodos tienen un techo de calidad y recorre las técnicas modernas de **Super-Resolución de Imagen Única** (Single Image Super-Resolution, **SISR**): desde "aprender de ejemplos" (RAISR), pasando por las redes convolucionales y los GAN (Real-ESRGAN), hasta los **Transformers** (SwinIR / Swin2SR).

Índice

1. Recordatorio: el problema del reescalado
2. Interpolación clásica: la matemática y su techo
3. Formulación de la Super-Resolución
4. El modelo de degradación (degradation pipeline)
5. Super-Resolución basada en ejemplos: del diccionario a RAISR
6. Super-Resolución con redes convolucionales (CNN)
7. El salto perceptual: SRGAN, ESRGAN y Real-ESRGAN
8. Transformers para imágenes: ViT, SwinIR y Swin2SR
9. Métricas de calidad y el dilema percepción–distorsión
10. Más allá de la guía: difusión, escala arbitraria y otras familias
11. Notas prácticas (Colab / GPU T4)
12. Referencias

1. Recordatorio: el problema del reescalado

💬 **En palabras sencillas.** Ampliar una imagen no es "estirla": es **inventarse píxeles que nunca existieron**. Al pasar de 100×100 a 400×400 píxeles, de cada 16 píxeles del resultado solo 1 viene de la foto original; los otros **15 hay que adivinarlos**. Y no hay una única respuesta correcta.

🎯 **Ejemplo para la audiencia.** Imagina una foto borrosa donde se ve una mancha oscura a lo lejos. ¿Es un perro? ¿Una maleta? ¿Una persona agachada? Todas esas escenas, fotografiadas de lejos, producirían **la misma mancha**. Ampliar la imagen obliga a *elegir* una de esas posibilidades. Cada método de ampliación es, simplemente, una forma distinta de tomar esa decisión: la interpolación decide "que todo quede suave"; la IA decide "que parezca una foto real".

Ampliar (*upscaling*) una imagen significa estimar los valores de píxeles que **no existían** en la imagen original. Si una imagen de baja resolución (LR) de tamaño $W \times H$ se amplía un factor s , hay que generar $s^2 \cdot W \cdot H - W \cdot H$ píxeles nuevos. El reescalado es, por tanto, un **problema mal planteado** (*ill-posed*): para una misma imagen

LR existen infinitas imágenes de alta resolución (HR) que, al reducirse, producirían esa LR. Cualquier método de ampliación no es más que una forma de **elegir una** de esas infinitas soluciones.

- La **interpolación clásica** elige asumiendo que la imagen es *suave* (los píxeles nuevos son combinaciones de los vecinos). Es una *prior* matemática fija — una "regla de sentido común" escrita de antemano.
- La **super-resolución aprendida** elige usando una *prior* aprendida de datos reales: "las imágenes naturales tienen bordes, texturas, patrones de piel/madera/pelo...". Por eso puede "alucinar" detalle plausible que la interpolación nunca generaría.

🧠 Frase para la diapositiva: "Ampliar no es recuperar información: es apostar. La diferencia entre métodos es quién hace la apuesta y con qué experiencia."

2. Interpolación clásica: la matemática y su techo

💡 **En palabras sencillas.** Los cuatro métodos clásicos hacen lo mismo con distinta sofisticación: rellenan cada píxel nuevo **mezclando el color de sus vecinos**, con una receta fija que no cambia nunca, sea la foto de un gato o de un texto.

🎨 **Analogía de la pintura.** Piensa en cada píxel nuevo como un bote de pintura que hay que rellenar mirando los botes de alrededor:

- **Nearest** = *fotocopiar el vecino más cercano*. Rápido y sin sorpresas, pero el resultado parece hecho de piezas de Lego (bloques).
- **Bilineal** = *mezclar los 4 botes vecinos a partes proporcionales*. Desaparecen los bloques... pero al mezclar tanto, todo queda borroso.
- **Bicúbica** = *un pintor con más contexto*: mira 16 vecinos y traza curvas suaves en vez de rectas. El equilibrio estándar de la fotografía.
- **Lanczos** = *el perfeccionista*: usa la fórmula matemáticamente "ideal" y consigue los bordes más nítidos, pero a veces se pasa y deja un "eco" o halo alrededor de los bordes fuertes (como cuando subes demasiado la nitidez en el móvil).

El notebook compara cuatro métodos. Todos son **convoluciones con un núcleo separable** $k(x,y) = k(x) \cdot k(y)$, así que basta con definir el núcleo 1D $k(x)$ (la distancia x se mide en unidades de píxel de la rejilla origen). Ampliar un factor s equivale a muestrear la señal reconstruida $\sum_n I[n] \cdot k(x - n)$ en las nuevas posiciones fraccionarias.

Nearest-Neighbor

Asigna a cada píxel destino el valor del píxel origen más cercano. Su núcleo es la **función caja** (*box*):

$$k_{\text{near}}(x) = \begin{cases} 1 & |x| < 0.5 \\ 0 & \text{en otro caso} \end{cases}$$

Soporte de 1 píxel, coste **O(1)** por muestra. En frecuencia su respuesta es un **sinc** muy ancho: deja pasar mucho *aliasing*, de ahí los "escalones" y bloques. Preserva bordes duros sin halos y **no inventa valores**: por eso es el correcto para máscaras, mapas de etiquetas o *pixel art*, donde mezclar colores sería un error (un píxel "medio gato, medio fondo" no significa nada en una máscara).

Bilineal

Interpolación lineal en X y luego en Y $\Rightarrow$ media ponderada de los **4** vecinos, con pesos proporcionales al área opuesta. Núcleo **triangular** (*tent*), soporte de 2 píxeles:

$$k_{\text{lin}}(x) = \max(0, 1 - |x|)$$

Es C^0 (continua pero con derivada discontinua). Atenúa las altas frecuencias de forma suave $\rightarrow$ elimina casi todo el *aliasing* pero **emborrona**. Coste bajo (4 muestras por píxel).

Bicúbica

Núcleo cúbico por tramos de **Keys**, soporte de 4 píxeles (vecindario **4x4** = 16 muestras en 2D). Con el parámetro **a** (habitualmente **a** = **-0.5**, que hace el núcleo tangente al **sinc** en el origen):

$$k_{\text{cub}}(x) = \begin{cases} (a+2)|x|^3 - (a+3)|x|^2 + 1 & |x| \leq 1 \\ a|x|^3 - 5a|x|^2 + 8a|x| - 4a & 1 < |x| < 2 \\ 0 & |x| \geq 2 \end{cases}$$

Es C^1 (derivada continua) y su respuesta en frecuencia es más plana en la banda de paso y cae más rápido que la bilineal $\rightarrow$ **mejor preservación de detalle** sin escalones. Con **a** = **-0.75** (convención de OpenCV) enfatiza algo más los bordes. Es el estándar de facto en fotografía.

Lanczos

Aproxima el reconstructor ideal (**sinc**) recortándolo con una **ventana sinc(x/a)** para hacerlo de soporte finito (**a** = 2 o 3; en OpenCV **INTER_LANCZOS4** usa **a** = 4):

$$k_{\text{lanczos}}(x) = \begin{cases} \text{sinc}(x) \text{ sinc}(x/a) & |x| < a \\ 0 & |x| \geq a \end{cases}, \quad \text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

El **sinc** es el filtro paso-bajo ideal (recuperación perfecta de una señal de banda limitada, teorema de Nyquist–Shannon), así que Lanczos da la **banda de paso más plana** y los **bordes más nítidos** de los cuatro. El precio: sus lóbulos negativos generan *overshoot* $\rightarrow$ **halos** (*ringing*) cerca de bordes de alto contraste, y su soporte grande lo hace el más costoso.

Comparativa rápida

Método	Soporte 1D	Vecinos 2D	Continuidad	Efecto dominante	En una frase
Nearest	1 px	1	discontinua	bloques / aliasing	"fotocopia al vecino"
Bilineal	2 px	4	C^0	emborronamiento	"media de los 4 de al lado"
Bicúbica	4 px	16	C^1	equilibrio nitidez/suavidad	"curva suave con 16 vecinos"
Lanczos	2a px	(2a) ²	C^∞ a tramos	nitidez + posible ringing	"el ideal matemático, con eco"

El techo común

💡 **En palabras sencillas.** Por sofisticada que sea la mezcla, **mezclar pintura solo produce colores que ya estaban en los botes**. Ninguna receta fija puede recuperar la letra pequeña de un cartel que en la foto original ocupaba 3 píxeles: esa información **ya no está**. Es como intentar leer la fotocopia de una fotocopia de una fotocopia jugando solo con el brillo y el contraste — el texto perdido no vuelve.

Todos son **filtros lineales de convolución con un núcleo fijo**. La salida es una combinación lineal de píxeles de entrada:

$$I_{HR}(x, y) = \sum_{i, j} k(i, j) I_{LR}(x_i, y_j)$$

Como k no depende del contenido de la imagen, **no puede añadir frecuencias nuevas**: solo redistribuye la información existente. Matemáticamente, la energía en altas frecuencias que se perdió al reducir la imagen **no se puede recuperar** con un filtro lineal. De ahí el emborronamiento inevitable. Romper ese techo exige una *prior* no lineal y dependiente del contenido — un método que "sepa" qué aspecto tienen las cosas del mundo real. Eso es lo que aporta el aprendizaje.

📖 Lectura recomendada sobre Lanczos: <https://mazzo.li/posts/lanczos.html>

3. Formulación de la Super-Resolución

💡 **En palabras sencillas.** La super-resolución aprendida busca entrenar un "restaurador": una función que recibe la foto pequeña y devuelve su mejor estimación de la foto grande. ¿Cómo se entrena a un restaurador? Igual que a un aprendiz de taller: enseñándole miles de parejas de **"antes y después"** para que aprenda el camino de vuelta.

🎯 **La pregunta clave** que hay que dejar caer en la presentación: "¿Y de dónde sacamos miles de parejas de la misma foto en versión mala y versión buena?" — porque nadie tiene la misma escena fotografiada dos veces, perfectamente alineada, con dos cámaras distintas. La respuesta (sorprendentemente simple) es la siguiente sección.

La SISR busca una función F que, dada una imagen LR, estime la HR:

$$\hat{I}_{HR} = F(I_{LR}; \theta)$$

donde θ son parámetros aprendidos minimizando una pérdida sobre un conjunto de pares (I_{LR}, I_{HR}) :

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{n=1}^N \mathcal{L}(F(I_{LR}^{(n)}; \theta), I_{HR}^{(n)})$$

La pieza clave es: **¿de dónde salen esos pares de entrenamiento?** Casi nunca se tienen fotos reales del mismo objeto a dos resoluciones perfectamente alineadas. La solución —exactamente la intuición de "downgradeear imágenes buenas"— es el modelo de degradación.

4. El modelo de degradación (degradation pipeline)

🗨 **En palabras sencillas.** Como no existen parejas "foto mala / foto buena" del mundo real, **las fabricamos nosotros**: cogemos fotos buenísimas y las estropeamos a propósito (las achicamos, las emborronamos, les metemos ruido, las comprimimos). Así tenemos el "antes" (la estropeada) y el "después" (la original), y el modelo aprende a deshacer el estropicio.

🎯 **Analogía del profesor.** Es como un profesor que fabrica sus propios exámenes: parte de la solución correcta (la foto buena), la "esconde" (la foto degradada) y entrena al alumno a reconstruirla. Como el profesor conoce la solución exacta, puede corregir al alumno con total precisión millones de veces.

🎯 **Y la versión realista:** una foto que circula por internet no se ha estropeado una sola vez — es como un **meme reenviado mil veces por WhatsApp**: cada reenvío la comprime y emborrona un poco más. Por eso los modelos modernos (Real-ESRGAN) estropean las fotos de entrenamiento **varias veces seguidas y de formas variadas**: para que el alumno haya visto de todo.

Se parte de imágenes HR de alta calidad y se generan sus versiones LR aplicando una degradación conocida. El modelo clásico ("bicubic degradation") es:

$$I_{LR} = (I_{HR} * k) \downarrow_s$$

es decir, **desenfoque** (convolución con un núcleo k) seguido de **submuestreo** por un factor s . El modelo realista (BSRGAN / Real-ESRGAN) lo amplía con una cadena más rica:

$$I_{LR} = \left[(I_{HR} * k) \downarrow_s + n \right]_{\text{JPEG}}$$

añadiendo **ruido** n (gaussiano, de Poisson, del sensor) y **artefactos de compresión JPEG**, a menudo aplicados varias veces ("high-order degradation") para imitar imágenes del mundo real (capturas, fotos antiguas, vídeo recomprimido).

En el notebook, la función `degrade()` implementa la versión simple (achicar + reampliar con bicúbica) para ilustrar el concepto y generar el par de entrenamiento de la "tabla".

Anatomía de cada bloque (qué imita cada "estropicio")

- **Desenfoque** k → imita la **óptica** (lente desenfocada, movimiento de la cámara). No es un solo núcleo: se muestrea de una familia (gaussiano isotrópico y **anisotrópico**, más núcleos *generalized/plateau* Gaussian). Modela el *point spread function* del sistema óptico.
- **Submuestreo** $\downarrow_s$ → imita la **pérdida de resolución**. Se elige aleatoriamente entre *area*, bilineal y bicúbico; cada modo deja una "firma" de *aliasing* distinta que el modelo aprende a revertir.
- **Ruido** n → imita el **sensor con poca luz** (el granulado de las fotos nocturnas). Mezcla de gaussiano (aditivo) y de Poisson (dependiente de la señal).
- **JPEG** → imita **internet**: la compresión introduce bloques 8×8 y halos alrededor de los bordes (cuantización DCT). Es el defecto más común en imágenes descargadas.

High-order degradation (Real-ESRGAN)

El salto clave de Real-ESRGAN es aplicar la cadena **dos veces en serie** (*second-order*):

$$I_{LR} = \mathcal{D}_2(\mathcal{D}_1(I_{HR})), \quad \mathcal{D}_i = \left[(\cdot * k_i) \downarrow_{s_i} + n_i \right]_{\text{JPEG}_i}$$

Una sola pasada no captura imágenes que ya venían degradadas (una foto de internet reescalada y recomprimida varias veces — el "meme reenviado"). Encadenar dos degradaciones aleatorias amplía enormemente el espacio de degradaciones "vistas". Además se añade un **filtro sinc** al final para modelar el *ringing* y el *overshoot* típicos de reescalados previos.

Por qué importa: el modelo aprendido solo sabe deshacer las degradaciones que vio en entrenamiento — como un cerrajero que solo sabe abrir los modelos de cerradura con los que practicó. Un modelo entrenado solo con "bicubic degradation" funciona mal con fotos reales ruidosas; por eso Real-ESRGAN invierte tanto esfuerzo en una degradación realista. El riesgo opuesto también existe: una degradación **demasiado** agresiva hace que el modelo alucine y sobre-suavice imágenes que en realidad estaban limpias.

👤 BSRGAN (degradación aleatoria): <https://arxiv.org/abs/2103.14006> · Real-ESRGAN (high-order): <https://arxiv.org/abs/2107.10833>

5. Super-Resolución basada en ejemplos: del diccionario a RAISR

💡 **En palabras sencillas.** La primera idea "de aprendizaje" es la más intuitiva: montar un **diccionario gigante** de casos vistos. *"Cuando alrededor de un píxel veo este patrón, el resultado bueno era este color."* Es exactamente la "tabla de vecindarios" del notebook. La idea funciona... hasta que haces las cuentas.

🌀 **El muro, con números — la analogía del candado.** ¿De dónde sale el 256? Cada píxel de una imagen en escala de grises se guarda con **8 bits**, y $2^8 = 256$: su brillo puede tomar 256 valores, de 0 (negro) a 255 (blanco). Cada casilla de la tabla es una combinación exacta de los brillos de los 8 vecinos — como la combinación de un **candado de 8 ruedas con 256 posiciones cada una**. Igual que un candado de 4 ruedas con 10 dígitos tiene $10^4 = 10.000$ combinaciones, aquí salen $256 \times 256 \times \dots \times 256$ (8 veces) $= 256^8 \approx 18$ **trillones** de patrones posibles — millones de veces más entradas que granos de arena hay en la Tierra. Y como casi ninguna combinación se repite entre fotos, la tabla nunca puede llenarse: cuando la consultas, casi siempre está **vacía justo en la casilla que necesitas**.

🌀 **El truco de RAISR = ordenar el armario.** En vez de una percha para cada prenda posible (imposible), organizas el armario en **unos 200 cajones por tipo**: "borde vertical fuerte", "borde diagonal suave", "textura sin dirección clara"... Y en cada cajón guardas **una única regla de retoque** aprendida de todos los ejemplos que cayeron ahí. De trillones de entradas pasamos a ~200 cajones — y de repente la idea es viable, rapidísima y cabe en unos pocos KB.

Es la familia a la que pertenece, conceptualmente, la "tabla de vecindarios" del notebook.

Example-based SR (Freeman et al., 2002)

Se construye una base de datos de **parches** (*patches*) emparejados LR↔HR. Para ampliar, por cada parche LR de la imagen de entrada se busca el parche LR más parecido del diccionario y se pega su contraparte HR. Funciona, pero:

- La base de datos es enorme y la búsqueda del vecino más cercano es lenta.
- **La maldición de la dimensionalidad:** el número de configuraciones posibles de un vecindario crece exponencialmente con su tamaño. Con 8 vecinos en escala de grises ya hay $256^8 \approx 1.8 \cdot 10^{19}$ combinaciones; en color es astronómico. La tabla nunca se puede llenar ni almacenar → se queda vacía justo donde la consultas.

De dónde sale exactamente el 256^8

La base y el exponente tienen cada uno su origen:

- $256 = 2^8$ es el número de niveles que puede tomar **un** píxel de 8 bits en escala de grises (de 0 a 255). No es una elección del método: es cómo se almacenan las imágenes estándar.
- El **exponente 8** es el número de vecinos considerados: el anillo que rodea al píxel central en un vecindario 3×3 ($3 \times 3 = 9$ píxeles, menos el central = 8).
- Como cada vecino toma su valor **independientemente** de los demás, el total de patrones es el producto: $256 \times 256 \times \dots \times 256$ (8 veces) = $256^8 \approx 1.8 \cdot 10^{19}$.

De ahí que la tabla escale de forma explosiva al ampliar el contexto o añadir color:

Vecindario	Vecinos	En gris (8 bits)	En color (RGB)
3×3 (1 anillo)	8	$256^8 \approx 1.8 \cdot 10^{19}$	$(256^3)^8 = 256^{24} \approx 6.3 \cdot 10^{57}$
5×5 (2 anillos)	24	$256^{24} \approx 6.3 \cdot 10^{57}$	$(256^3)^{24} \approx 10^{173}$

En color cada píxel tiene $256^3 \approx 16,7$ millones de valores posibles (256 por canal R, G y B), así que la base pasa de 256 a 16,7 millones por vecino. Nótese además que **más contexto empeora el problema**: mirar más vecinos daría mejores decisiones, pero hace la tabla aún más imposible de llenar.

Esta cuenta es exactamente la que ejecuta la celda del notebook con la función `sci()`. Y es también la palanca del truco de cuantizar: con $Q = 4$ cajas por vecino y el vecindario 3×3 completo (9 píxeles), la tabla del notebook tiene $4^9 = 262.144$ entradas — de trillones a algo que cabe de sobra en memoria. El precio es perder matiz: todos los brillos entre 0 y 63 caen en la misma caja.

Sparse coding / dictionary learning (Yang et al., 2010)

En vez de guardar todos los parches, aprende un **diccionario compacto** y representa cada parche como combinación *sparse* de sus átomos. Comprime el problema, pero sigue siendo lento.

RAISR (Google, 2016) — la idea "apañada"

Rapid and Accurate Image Super-Resolution. En lugar de una tabla con todos los colores, clasifica (*hashing*) cada parche en un número pequeño de **cubos (buckets)** — los "cajones del armario" — según características de su **gradiente local**, y aprende un **filtro lineal óptimo por cubo**.

El hash por gradiente (cómo decide a qué cajón va cada parche). No mira el color crudo, sino **la forma del borde**: para el parche se calcula el **tensor de estructura** (la matriz de covarianza de los gradientes ∇I ponderada por una gaussiana). Sus dos autovalores $\lambda_1 \geq \lambda_2$ y el autovector principal dan tres descriptores, cada uno cuantizado en pocos niveles:

- **Ángulo** θ — ¿hacia dónde apunta el borde? → p. ej. 24 cubos.
- **Fuerza** (*strength*) $\approx \sqrt{\lambda_1}$ — ¿es un borde marcado o suave? → p. ej. 3 cubos.
- **Coherencia** $(\sqrt{\lambda_1} - \sqrt{\lambda_2}) / (\sqrt{\lambda_1} + \sqrt{\lambda_2}) \in [0, 1]$ — ¿es un borde limpio o textura desordenada? → 3 cubos.

Eso da del orden de $24 \times 3 \times 3 = 216$ cubos ($\times$ el factor de escala), **muchísimos menos** que las 256^8 de la tabla ingenua, pero **informados por la geometría del borde** en vez de por el color crudo.

Aprender el filtro de cada cubo. Por cada cubo se resuelve un problema de **mínimos cuadrados**: se busca el filtro h que mejor mapea los parches LR de ese cubo a su píxel HR central,

$$h^* = \arg \min_h \sum_{p \in \text{cubo}} (h^\top p_{LR} - p_{HR})^2 = (A^\top A)^{-1} A^\top b$$

donde las filas de A son los parches LR y b los píxeles HR objetivo. Es la **solución óptima cerrada** (ecuación normal), no un descenso de gradiente.

Inferencia (muy barata): para cada píxel $\rightarrow$ gradiente $\rightarrow$ índice de cubo $\rightarrow$ producto escalar con su filtro. Google añade además *blending* con **census/coherencia** para suavizar transiciones y evitar artefactos. Resultado: calidad cercana a las CNN de su época, **$\sim 100\times$ más rápido** que ellas y con un modelo de unos pocos KB.

RAISR es prácticamente la versión funcional de la "tabla de vecindarios con cajas". Dos ideas lo hacen viable frente a la tabla ingenua: (1) **cuantizar por geometría del gradiente** en vez de por color, y (2) **aprender un filtro lineal por cubo** en lugar de memorizar un color. La mini-implementación del notebook (cuantizar el vecindario 3×3 en Q niveles y aprender el color medio por caja) es un RAISR de juguete: usa (1) de forma tosca y sustituye (2) por la media, para mostrar por qué la agrupación en pocas cajas es lo que rompe el muro de la tabla. *(Nota para la demo: en el notebook entrenamos y aplicamos sobre la misma imagen — por eso queda tan bien; está "memorizando", no generalizando.)*

RAISR: <https://arxiv.org/abs/1606.01299>

6. Super-Resolución con redes convolucionales (CNN)

En palabras sencillas. El siguiente salto es dejar de guardar casos y pasar a **aprender una intuición**. Una red neuronal no memoriza una tabla: comprime la experiencia de millones de ejemplos en sus pesos, igual que un médico veterano no recuerda a cada paciente que ha visto, pero **reconoce el patrón** en un paciente nuevo que no ha visto jamás. Eso es *generalizar* — justo lo que a la tabla le era imposible.

Tres ideas de esta etapa, en versión llana:

- **Pixel shuffle** (ESPCN): en vez de ampliar la imagen y luego retocarla, la red prepara 16 "mini-versiones" de cada zona y luego las **entrelaza como las piezas de un puzzle** para formar la imagen grande. Mucho más barato y sin artefactos.
- **Aprendizaje residual** (VDSR): en vez de redibujar toda la imagen, la red actúa como un **corrector de textos**: no reescribe el documento, solo marca lo que falta o sobra (el detalle fino). Corregir es más fácil que redactar de cero.
- **Atención de canal** (RCAN): la red incorpora un **ecualizador automático**, como el de un equipo de música: para cada imagen decide qué "frecuencias" (canales con detalle fino) subir y cuáles bajar.

La idea de "aprender la *prior* de las imágenes naturales" cristaliza con el deep learning. Una CNN **no guarda una tabla**: guarda una función comprimida en sus pesos que **generaliza** a vecindarios nunca vistos. Hitos:

Modelo	Año	Aportación principal
SRCNN	2014	Primera CNN para SR. 3 capas: extracción de parches, mapeo no lineal, reconstrucción. Demostró que una red supera a la interpolación.
FSRCNN	2016	Procesa en resolución LR y amplía al final → mucho más rápido.
ESPCN	2016	Introduce el sub-pixel convolution / pixel shuffle : la red aprende s^2 mapas y los reordena para ampliar, sin interpolar antes. Estándar hoy.
VDSR	2016	Red muy profunda (20 capas) + aprendizaje residual (predice solo el detalle que falta).
EDSR	2017	Bloques residuales sin BatchNorm; ganador del reto NTIRE. Gran calidad en PSNR.
RCAN	2018	Atención de canal + redes muy profundas (400+ capas).

Los mecanismos clave en detalle

SRCNN (3 capas). Formaliza la SR como tres operaciones aprendidas de golpe: 9×9 conv (*extracción de parches* → mapa de features), 1×1 conv (*mapeo no lineal* LR→HR en el espacio de features) y 5×5 conv (*reconstrucción*). Trabaja sobre la imagen **ya interpolada** con bicúbica (por eso es lento: opera a resolución HR).

FSRCNN — procesar en LR. Estructura en "reloj de arena": *shrinking* 1×1 para reducir canales, varias conv pequeñas de mapeo, *expanding*, y al final una **deconvolución** que hace el *upscaling*. Al trabajar casi todo en resolución LR es **decenas de veces más rápido** que SRCNN. (Intuición: es más barato pensar sobre la foto pequeña y ampliar al final que ampliar primero y pensar sobre la grande.)

ESPCN — sub-pixel convolution / pixel shuffle. En vez de interpolar antes, la red produce s^2 mapas de features a resolución LR y los **reordena** (*PixelShuffle*) en un solo mapa s veces mayor:

$$PS : (C \cdot s^2) \times H \times W \longrightarrow C \times (sH) \times (sW)$$

Cada bloque de s^2 canales se "despliega" en una vecindad $s \times s$ — el entrelazado del puzzle. Ventajas frente a la deconvolución: **sin artefactos de tablero** (*checkerboard*) y cómputo en LR. Es el módulo de *upscaling* estándar hoy (lo usan SRGAN, EDSR, etc.).

VDSR — aprendizaje residual. Con 20 capas, en vez de predecir la HR completa predice solo el **residuo** (el detalle que falta): $\hat{I}_{HR} = I_{bicúbica} + R(I_{bicúbica})$. La red modela una señal casi nula salvo en los bordes → converge mucho mejor. Requirió *gradient clipping* para permitir tasas de aprendizaje altas en una red tan profunda.

EDSR — limpiar el bloque residual. Quita la **BatchNorm** de los bloques residuales (en SR, BN normaliza y "aplana" el rango de features, perjudicial cuando la salida debe ser fiel al color de entrada). Eso libera memoria para hacer la red más ancha/profunda, con **residual scaling** (multiplicar la rama residual por ~ 0.1) para estabilizar el entrenamiento.

RCAN — atención de canal (CA) + RIR. Introduce *Residual-in-Residual* (residuos anidados con *long skip connections*) para poder apilar 400+ capas, y **atención de canal**: por cada canal se calcula un descriptor global (*global average pooling*) y una pequeña red *sigmoid* produce un peso $\in [0, 1]$ que **recalibra** ese canal:

$$s = \sigma(W_2 \delta(W_1 z)), \quad z_c = \frac{1}{HW} \sum_{i,j} x_c(i,j), \quad \hat{x}_c = s_c x_c$$

La red aprende a **dar más peso a los canales que codifican alta frecuencia** (el detalle fino) y a atenuar los de baja información. (RCAB = bloque residual + esta rama de atención.)

Funciones de pérdida. Las primeras CNN minimizan el error píxel a píxel:

$$\mathcal{L}_1 = \|I_{HR} - \hat{I}_{HR}\|_1 \quad \mathcal{L}_2 = \|I_{HR} - \hat{I}_{HR}\|_2^2$$

💡 **En palabras sencillas.** Una función de pérdida es la **nota del examen**: compara la imagen que produce la red con la original y le dice cuánto se ha equivocado, píxel a píxel. La red entrena para sacar la mejor nota posible. L1 y L2 son dos profesores con criterios de corrección distintos:

🌀 **La analogía de las multas de tráfico.**

- **L1** (error absoluto) es una multa **proporcional**: pasarte 10 km/h cuesta 10, pasarte 40 cuesta 40. Cada fallo pesa exactamente lo que mide.
- **L2** (error al cuadrado) es una multa **que se dispara**: pasarte 10 cuesta 100 (10^2), pero pasarte 40 cuesta 1.600 (40^2). Un error grande cuesta muchísimo más que varios errores pequeños que sumen lo mismo.

La consecuencia sobre la imagen: el "alumno" entrenado con L2 le tiene **pánico a los errores grandes**, así que juega sobre seguro: ante la duda, pone un valor intermedio y grisáceo — resultado borroso pero que nunca falla estrepitosamente. Con L1 el castigo por arriesgar es proporcional, no explosivo, así que el alumno se atreve un poco más con los bordes y el detalle. (En términos estadísticos: minimizar L2 empuja la predicción hacia la **media** de las soluciones plausibles, minimizar L1 hacia la **mediana** — y la mediana es menos borrosa que la media. Por eso los modelos modernos como EDSR entrenan con L1.) Aun así, **ambos profesores corrigen letra a letra**: ninguno valora si la imagen "parece real" en conjunto — esa limitación es la que resuelve el siguiente salto.

Maximizan PSNR pero tienden a dar imágenes **suaves** (la media de todas las HR plausibles es borrosa). Resolver eso lleva al siguiente salto.

7. El salto perceptual: SRGAN, ESRGAN y Real-ESRGAN

💡 **En palabras sencillas — por qué las CNN "prudentes" emborronan.** Si diez testigos describen a un sospechoso y el dibujante hace **la media de los diez retratos**, sale una cara borrosa y genérica que no se parece a nadie — pero que "de media" no se equivoca mucho con ninguno. Eso hace exactamente una red entrenada con error píxel a píxel: como hay muchas imágenes nítidas posibles, la apuesta más segura es **el promedio de todas**, que es borroso. Arriesgar (dibujar una textura nítida concreta) penaliza más si falla por poco. El óptimo matemático es, literalmente, "no arriesgar".

🌀 **La solución GAN = el falsificador y el experto.** Se entrenan dos redes que compiten: un **falsificador** (el generador, que amplía imágenes) y un **experto de museo** (el discriminador, que intenta distinguir las fotos reales de las generadas). Cada vez que el experto pilla una falsificación, el falsificador aprende y mejora; y cada vez que el falsificador le cuela una, el experto afina el ojo. Tras millones de rondas, las "falsificaciones" son indistinguibles de fotos reales: texturas de piel, pelo, ladrillo... **nítidas y creíbles**, aunque no idénticas píxel a píxel al original.

🎯 **Pérdida perceptual, en una frase:** en vez de comparar las dos imágenes píxel a píxel (letra a letra), se compara **la impresión general** — como comparar dos interpretaciones de la misma canción: no importa que cada nota caiga en el mismo milisegundo, sino que *suene* igual.

Por qué las pérdidas L1/L2 emborronan (versión técnica)

Para una LR dada hay muchas HR plausibles (§1). Minimizar el error medio empuja a **F** hacia la **media** de todas ellas, y la media de texturas alineadas de forma distinta es **borrosa**. Un detalle nítido pero ligeramente desplazado penaliza *más* en L2 que un promedio suave, así que el óptimo por MSE es literalmente "no arriesgar". Romperlo exige una señal que premie el **realismo** aunque el detalle no coincida píxel a píxel. Eso son la pérdida perceptual y la adversaria.

SRGAN (2017)

Introduce dos ideas para imágenes **perceptualmente** nítidas en vez de óptimas en PSNR:

- **Pérdida perceptual** (*perceptual / content loss*): en vez de comparar píxeles, compara *features* de una red **VGG-19** preentrenada, $\mathcal{L}_{perc} = \|\phi(I_{HR}) - \phi(\hat{I}_{HR})\|^2$, donde ϕ es la activación de una capa intermedia. Premia que las **texturas y estructuras** "parezcan" correctas, no que coincida cada píxel.
- **Pérdida adversaria** (GAN): un **discriminador** **D** aprende a distinguir HR reales de generadas, y el generador **G** aprende a engañarlo. El discriminador es, en esencia, el "juez que detecta inventos que no parecen reales".

$$\mathcal{L}_G = \underbrace{\mathcal{L}_{perceptual}}_{\text{textura realista}} + \lambda \underbrace{\mathcal{L}_{adv}}_{\text{engañar a } D} + \eta \underbrace{\mathcal{L}_1}_{\text{fidelidad de color}}$$

Arquitectura. El generador es una red residual (bloques *conv-BN-PReLU*) que trabaja en LR y amplía al final con **sub-pixel conv** (§6). El discriminador es una CNN tipo VGG que termina en una probabilidad "real/falsa". El término L1/perceptual evita que el color derive mientras la parte adversaria añade el detalle de alta frecuencia.

ESRGAN (2018)

Mejora SRGAN en tres frentes:

- **Bloque RRDB** (*Residual-in-Residual Dense Block*): combina conexiones **densas** (cada conv ve la salida de todas las anteriores) con residuos anidados y **sin BatchNorm** (que en SR introduce artefactos). Usa **residual scaling** $\beta \approx 0.2$ en cada rama para estabilizar redes muy profundas.
- **Discriminador relativista** (RaGAN): en lugar de estimar "¿es real?" (absoluto), estima "**¿es esto más realista que aquello?**" (relativo): $D(x_r, x_f) = \sigma(C(x_r) - \mathbb{E}[C(x_f)])$. Un juez que compara siempre da un feedback más útil que uno que solo dice sí/no. Da gradientes más informativos y texturas más agudas.
- **Pérdida perceptual antes de la activación** (*pre-activation VGG features*): las features pre-ReLU conservan más información de brillo y bordes → texturas más realistas.

Real-ESRGAN (2021) — el que usa el notebook

Adapta ESRGAN al **mundo real** (mismo generador RRDB) combinando:

1. **Degradación de alto orden** (§4): desenfoque + ruido + JPEG + **sinc** aplicados **dos veces en serie** — el entrenamiento con "memes reenviados mil veces" — para que el modelo sepa restaurar fotos reales degradadas, no solo "bicubic".
2. Un **discriminador U-Net con spectral normalization**. A diferencia del discriminador global de ESRGAN (que da un veredicto único para toda la imagen), el U-Net emite un **mapa de realismo por píxel** — un experto que no solo dice "es falso", sino que **señala con el dedo qué zonas** lo delatan. La *spectral norm* (limitar la norma espectral de los pesos) **estabiliza** el entrenamiento y frena artefactos cuando la degradación es agresiva.

Es el estándar práctico para *upscaling* de fotos y arte (existe **Real-ESRGAN-anime**, con un generador más ligero, para ilustración). Pesos x4 listos para usar; soporta *tiling* para imágenes grandes.

🗣 ESRGAN: <https://arxiv.org/abs/1809.00219> · Real-ESRGAN: <https://arxiv.org/abs/2107.10833> · Repo: <https://github.com/xinntao/Real-ESRGAN>

8. Transformers para imágenes: ViT, SwinIR y Swin2SR

💬 **En palabras sencillas — qué aporta la "atención"**. Una CNN decide cada píxel mirando solo su barrio. Un Transformer puede mirar **toda la imagen a la vez**. Es la diferencia entre traducir una novela frase a frase y traducirla habiéndola leído entera: para entender el "él" de la página 200 hay que recordar quién apareció en la página 3. En una imagen pasa igual: para reconstruir bien un ladrillo borroso ayuda muchísimo ver que al otro lado de la foto hay ladrillos nítidos del mismo muro.

🎯 **El problema y el truco de Swin — la analogía de la cena**. Que todos "hablen con todos" es carísimo: en una sala con miles de invitados, el número de conversaciones posibles se dispara al cuadrado. **Swin** lo resuelve como una cena bien organizada: sienta a los invitados en **mesas de 8** (ventanas) donde solo se habla dentro de la mesa — barato. Y para que la información circule por toda la sala, **cada ronda recoloca las mesas** (shifted windows), como en una cena progresiva o un speed-dating: tu vecino de la ronda anterior lleva lo que oyó a la mesa nueva. En pocas rondas, todo el mundo se ha enterado de todo, sin que nadie haya tenido que hablar con miles de personas a la vez.

De ViT al problema de la super-resolución

El **Vision Transformer (ViT, 2020)** trocea la imagen en *parches*, los proyecta a *tokens* (los "invitados" de la analogía) y aplica **auto-atención**. Cada token genera *query*, *key* y *value*, y su salida es una media de los valores ponderada por la similitud query-key:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + B\right)V$$

La ventaja frente a la convolución es el **contexto global**: cada token puede "mirar" a todos los demás → modela dependencias de **largo alcance** (relaciona zonas lejanas), mientras que una conv solo ve su vecindario local.

El problema: con n tokens, QK^T es $n \times n \Rightarrow$ coste y memoria $\mathcal{O}(n^2)$. A resolución de imagen completa (miles de tokens) es inviable — la sala con todos hablando con todos.

Swin Transformer — atención por ventanas

Resuelve el coste partiendo la imagen en **ventanas** de $M \times M$ tokens (p. ej. $M = 8$) — las "mesas" — y calculando atención **solo dentro** de cada ventana (**W-MSA**). El coste pasa de cuadrático en todos los píxeles a **lineal**:

$$\Omega(\text{MSA}) = O((HW)^2) \rightarrow \Omega(\text{W-MSA}) = O(M^2 \cdot HW)$$

Pero ventanas fijas nunca dejan que la información cruce sus bordes. La solución son las **ventanas desplazadas** (*shifted windows*, **SW-MSA**): en capas alternas la rejilla de ventanas se desplaza $M/2$, de modo que tokens que antes estaban en ventanas distintas ahora comparten ventana — el cambio de mesa entre rondas. Se implementa con un **desplazamiento cíclico** + una **máscara de atención** (para no mezclar regiones no adyacentes) que mantiene el coste. Alternar bloques W-MSA y SW-MSA propaga el contexto por toda la imagen en pocas capas. Añade además un **sesgo de posición relativa** B (el término del softmax de arriba), que codifica dónde está cada token respecto a los otros dentro de la ventana.

SwinIR (2021)

Aplica Swin a la restauración de imagen (SR, *denoising*, *JPEG deartifacting*) con tres etapas:

1. **Extracción superficial**: una conv obtiene features iniciales.
2. **Extracción profunda**: una pila de **RSTB** (*Residual Swin Transformer Blocks*). Cada RSTB es una secuencia de **STL** (*Swin Transformer Layers*: W-MSA/SW-MSA + MLP con LayerNorm y conexiones residuales) cerrada por una conv y una **skip connection** larga. Mezclar atención (visión global) con conv (ojo para lo local) da lo mejor de ambos mundos.
3. **Reconstrucción**: fusiona features superficiales + profundas y amplía con **sub-pixel conv**.

Alcanza el estado del arte con **menos parámetros** que las CNN equivalentes (EDSR/RCAN), porque la atención captura dependencias que a la conv le costarían muchas capas.

Swin2SR (2022)

Actualiza SwinIR a **Swin Transformer V2**, que corrige inestabilidades al escalar:

- **Post-normalization + scaled cosine attention**: normaliza después del bloque y usa similitud **coseno** (en vez del producto escalar) para evitar que unos pocos tokens dominen el softmax — que unos pocos invitados acaparen toda la conversación — → entrenamiento estable en redes/resoluciones grandes.
- **Log-spaced continuous position bias**: aprende el sesgo de posición con una pequeña MLP sobre coordenadas en escala logarítmica, lo que permite **transferir** a ventanas de distinto tamaño que en entrenamiento.
- Modo **compressed-input** pensado para entradas con **JPEG** (deartifacting + SR a la vez).

Está integrado en 🤗 **transformers** (*Swin2SRForImageSuperResolution* + *Swin2SRImageProcessor*), por lo que se usa casi igual que un modelo de NLP — justo lo que hace el notebook.

Coste / memoria. La atención, aun por ventanas, consume bastante VRAM con imágenes grandes.

Recomendación: procesar **recortes** o trocear (*tiling*). En el notebook se aplica sobre un recorte pequeño (96 px → x4) por eso.

🤖 ViT: <https://arxiv.org/abs/2010.11929> · Swin: <https://arxiv.org/abs/2103.14030> · SwinIR: <https://arxiv.org/abs/2108.10257> · Swin2SR: <https://arxiv.org/abs/2209.11345> · Modelos: <https://huggingface.co/caidas>

9. Métricas de calidad y el dilema percepción–distorsión

💬 **En palabras sencillas.** ¿Cómo medimos qué ampliación es "mejor"? Hay dos filosofías que **no siempre coinciden**:

- **Fidelidad** (PSNR/SSIM): comparar con el original **píxel a píxel**, como corregir un dictado letra por letra.
- **Percepción** (LPIPS): preguntar si el resultado **parece** una foto real y se parece al original *a ojos de una persona*, aunque cada píxel no coincida.

🌀 **El dilema, con el retrato robot.** Un retrato robot "prudente" (suave, genérico) nunca se equivoca mucho, pero no ayuda a reconocer a nadie. Un retrato hiperrealista ayuda mucho más... pero el dibujante puede haber **inventado un lunar que no existía**. Blau & Michaeli demostraron que este compromiso es **matemáticamente inevitable**: no se puede ser a la vez máximamente fiel y máximamente realista.

🌀 **Por eso la elección depende del uso:** en una **radiografía o una prueba forense** quieres el método fiel (¡que no invente un tumor o una matrícula!); para **restaurar las fotos de la boda de tus abuelos** quieres el realista, aunque algún poro de la piel sea inventado.

Comparar métodos de SR no es trivial; conviene reportar varias métricas.

- **PSNR** (Peak Signal-to-Noise Ratio): derivado del MSE. Más alto = más fiel píxel a píxel. No correlaciona bien con la percepción humana (un resultado borroso puede tener alto PSNR).

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

- **SSIM** (Structural Similarity): compara **luminancia**, **contraste** y **estructura** en ventanas locales, combinando medias μ , varianzas σ^2 y covarianza $\sigma_{\{xy\}}$:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

Rango $[0, 1]$ (1 = idénticas), más cercano a la percepción que el PSNR porque penaliza cambios **estructurales**, no solo energéticos. Variante multi-escala: **MS-SSIM**.

- **LPIPS** (Learned Perceptual Image Patch Similarity): pasa ambas imágenes por una red profunda (AlexNet/VGG), y mide la **distancia L2 entre sus activaciones** capa a capa, con pesos aprendidos para alinearse con juicios humanos: $d = \sum_l \|w_l \odot (\phi_l(x) - \phi_l(y))\|^2$. En la práctica es "preguntarle a una red que ve como nosotros si las dos imágenes se parecen". **Más bajo = más parecido perceptualmente**. Hoy es la métrica perceptual de referencia.
- **NIQE / MANIQA / NRQM**: métricas **sin referencia** (*no-reference*), no necesitan la HR original; puntúan la "naturalidad" de la imagen a partir de estadísticas o de una red — como un catador que evalúa un vino sin haber probado "el original". Útiles para SR del mundo real donde **no hay ground truth** (justo el caso de Real-ESRGAN/Swin2SR sobre fotos reales).

El dilema percepción–distorsión (Blau & Michaeli, 2018): existe un compromiso **fundamental** entre fidelidad (PSNR/SSIM altos) y realismo perceptual (LPIPS bajo). No se pueden maximizar ambos a la vez. Por eso:

- Los modelos orientados a **PSNR** (EDSR, SwinIR-PSNR) dan resultados fieles pero más suaves.

- Los modelos **GAN/perceptuales** (ESRGAN, Real-ESRGAN) dan texturas nítidas y realistas, pero pueden "inventar" detalle que no estaba (peor PSNR). La elección depende del uso: fidelidad métrica (medicina, forense) vs. estética (fotografía, restauración).

🗣 The Perception-Distortion Tradeoff: <https://arxiv.org/abs/1711.06077> · LPIPS: <https://arxiv.org/abs/1801.03924>

10. Más allá de la guía: difusión, escala arbitraria y otras familias

💬 **En palabras sencillas.** La historia interpolación → RAISR → CNN → GAN → Transformer es la columna vertebral, pero el campo sigue: hoy el paradigma dominante son los **modelos de difusión**.

🌀 **Difusión = el escultor.** En vez de transformar la foto pequeña en una pasada, el modelo parte de un **bloque de ruido puro** (el mármol) y lo va **refinando poco a poco**, decenas de veces, usando la foto pequeña como referencia — como un escultor que retira material mirando de reojo al modelo. Resultado: el realismo más alto que existe... al precio de ser lento y de que a veces el escultor "se inspira demasiado" e inventa cosas que no estaban.

La progresión **interpolación → RAISR → CNN → GAN → Transformer** es la columna vertebral de la ISR, pero no agota el campo. Estas familias no se cubren en el notebook y algunas son **estado del arte hoy**. Vale la pena conocerlas para elegir la herramienta correcta.

Modelos de difusión (SR generativa) — el paradigma dominante actual

En lugar de una sola pasada, parten de **ruido** y lo *denoisan* iterativamente **condicionando en la LR**, muestreando de la distribución de HR plausibles en vez de promediarla (por eso esquivan el emborronamiento del MSE, §7 — el escultor elige *un* retrato concreto en vez de la media de los diez testigos). Formalmente aprenden a invertir un proceso de difusión:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_{\theta}(x_t, t, I_{LR}) \right) + \sigma_t z$$

- **SR3** (2021) — pionero, *iterative refinement* sobre píxeles.
- **SRDiff, Stable Diffusion x4 upscaler, LDM** — difusión en **espacio latente** (más barata).
- **StableSR / DiffBIR / PASD / SeeSR** (2023) — reutilizan un **prior de difusión preentrenado** (SD) para restaurar fotos reales degradadas.
- **SUPIR** (2024) — a gran escala y **guiado por texto** sobre SDXL; realismo altísimo, pero puede alucinar contenido no presente en la LR.
- **ResShift / SinSR** (2023–24) — difusión **eficiente** (pocos o un solo paso), atacando su mayor defecto: la lentitud (decenas de pasos de muestreo).

Trade-off vs. GAN: mejor realismo y menos artefactos, pero mucho **más lentos** en inferencia y con más tendencia a "inventar". La investigación reciente va sobre **acelerarlos** (destilación a 1–4 pasos).

Super-Resolución de escala arbitraria (representaciones implícitas)

- **LIIF** (2021), **LTE**, **CiaoSR** — aprenden una **función continua** de la imagen (*implicit neural representation*): dado un punto (**x,y**) continuo devuelven su color, así que amplían a **cualquier factor** (×2.7, ×5.3...) con un único modelo, no solo a los **s** entrenados. (Intuición: un zoom continuo, sin "escalones" de ×2/×4.)

Transformers más allá de Swin

- **HAT** (2023, *Hybrid Attention Transformer*) — combina atención de canal + de ventana + *overlapping cross-attention*; **líder actual en PSNR**.
- **IPT** (transformer preentrenado multi-tarea), **SRFormer**, **DAT**, **GRL**, **ELAN**.

State-space models / Mamba

- **MambalR** (2024) — usa *state-space models* de **complejidad lineal** como alternativa a la atención, buscando el contexto global del transformer sin su coste cuadrático.

Zero-shot / aprendizaje interno (sin dataset externo)

- **ZSSR** (2018) y **Deep Image Prior** — se entrenan con la **propia imagen de test**, explotando la recurrencia de parches a distintas escalas (una imagen se parece mucho a sí misma a distintos zooms). Útiles cuando no hay datos de entrenamiento o la degradación es desconocida.
- **KernelGAN** — estima el **kernel de degradación real** de la imagen (*blind SR*), que luego alimenta a un modelo de SR.

Restauración de caras (nicho muy práctico)

- **GFP-GAN**, **CodeFormer** (codebook VQ + transformer), **GPEN**, **PULSE** — usan un **prior facial** (típicamente StyleGAN) para reconstruir rostros con identidad plausible: un retratista que ha visto millones de caras y "sabe" cómo debe ser una ceja o un iris. Es la tecnología detrás de las funciones de "realzar caras" de muchas apps.

Fuera del alcance SISR (contexto)

- **Flow-based: SRFlow** (2020) — flujos normalizadores; modelan la distribución completa de HR y permiten muestrear salidas **diversas**.
- **Reference-based (RefSR): TTISR, MASA-SR, C2-Matching** — usan una **segunda imagen HR** de apoyo (misma escena/objeto) para transferir textura real: "chuletas" visuales.
- **Video SR: EDVR, BasicVSR / BasicVSR++ , RVRT** — explotan la **coherencia temporal** entre fotogramas: lo que en un fotograma está borroso puede verse nítido dos fotogramas después.

👤 SR3: <https://arxiv.org/abs/2104.07636> · StableSR: <https://arxiv.org/abs/2305.07015> · DiffBIR: <https://arxiv.org/abs/2308.15070> · ResShift: <https://arxiv.org/abs/2307.12348> · SUPIR: <https://arxiv.org/abs/2401.13627> · LIIF: <https://arxiv.org/abs/2012.09161> · HAT: <https://arxiv.org/abs/2205.04437> · MambalR: <https://arxiv.org/abs/2402.15648> · ZSSR: <https://arxiv.org/abs/1712.06087> · CodeFormer: <https://arxiv.org/abs/2206.11253> · SRFlow: <https://arxiv.org/abs/2006.14200> · BasicVSR++: <https://arxiv.org/abs/2104.13371>

11. Notas prácticas (Colab / GPU T4)

- **¿T4 suficiente?** Sí. Sus 16 GB de VRAM bastan para Real-ESRGAN x4 y Swin2SR sobre imágenes/recortes razonables. Activa la GPU en Colab: *Entorno de ejecución* → *Cambiar tipo de entorno* → *T4 GPU*.
- **Real-ESRGAN** y memoria: para imágenes grandes usa **tiling** (procesa la imagen por baldosas con solape, como alicatar una pared). El repo expone un parámetro **tile** para ello.

- **Swin2SR** es el más sensible a la VRAM (atención $\propto$ píxeles). Mantén la entrada por debajo de $\sim 512 \times 512$ px en una T4, o trocea. En el notebook se usa un recorte de 96 px $\rightarrow$ x4.
- **Espacio de color.** Mucha SR clásica opera sobre el canal de **luminancia (Y de YCbCr)**, donde vive el detalle que percibe el ojo, y amplía la crominancia por separado. (El ojo humano distingue mucho mejor el detalle de luces/sombras que el de color: por eso basta con afinar la luminancia.)
- **Factor de escala.** Los modelos están entrenados para un **s** concreto (x2, x4...). Usar el modelo correcto importa; encadenar x2 dos veces no equivale a un x4 entrenado.
- **Formato de entrada.** Da al modelo la imagen **LR real**; no le pases una ya ampliada con bicúbica (salvo modelos que lo esperan), o degradarás el resultado.

Chuleta de analogías para la presentación

Concepto	Analogía en una frase
Problema ill-posed	La mancha lejana: ¿perro, maleta o persona? Ampliar es apostar por una.
Interpolación	Rellenar píxeles mezclando la pintura de los vecinos con una receta fija.
Nearest / Bilineal / Bicúbica / Lanczos	Fotocopiar al vecino / media de 4 botes / pintor con contexto / el perfeccionista con eco.
El techo lineal	Mezclar pintura no crea colores nuevos; la fotocopia de la fotocopia no recupera el texto.
Degradation pipeline	El profesor que fabrica exámenes estropeando la solución a propósito.
High-order degradation	El meme reenviado mil veces por WhatsApp.
Tabla de vecindarios	Un diccionario con más entradas que granos de arena: siempre vacío donde consultas.
El 256 ⁸	Un candado de 8 ruedas (los vecinos) con 256 posiciones cada una (los niveles de gris, 2 ⁸).
RAISR	Ordenar el armario en ~ 200 cajones por tipo de borde, una regla de retoque por cajón.
CNN	El médico veterano: no memoriza pacientes, reconoce patrones (generaliza).
Aprendizaje residual	El corrector que solo marca lo que falta, no reescribe el texto.
Pixel shuffle	Entrelazar 16 mini-versiones como piezas de un puzzle.
Atención de canal	El ecualizador automático del equipo de música.
L1 vs L2	Multa proporcional vs multa que se dispara al cuadrado: con L2 la red juega sobre seguro (borroso).
Por qué L2 emborrona	La media de los retratos de diez testigos: borrosa pero "segura".
GAN	El falsificador de cuadros contra el experto del museo.
Pérdida perceptual	Comparar cómo <i>suen</i> a la canción, no nota a nota.

Concepto	Analogía en una frase
Discriminador U-Net	El experto que señala con el dedo qué zona delata la falsificación.
Atención (Transformer)	Traducir la novela habiéndola leído entera, no frase a frase.
Swin / shifted windows	Cena con mesas de 8 que se recolocan cada ronda: la información circula sin que todos hablen con todos.
PSNR vs LPIPS	Corregir el dictado letra a letra vs preguntar si "se parece".
Dilema percepción–distorsión	El retrato robot fiel pero soso vs el hiperrealista que inventa un lunar.
Difusión	El escultor que parte del ruido (mármol) y refina mirando la foto pequeña.

12. Referencias

Interpolación clásica

- Lanczos resampling — <https://mazzo.li/posts/lanczos.html>
- Keys, *Cubic convolution interpolation* (1981).

SR basada en ejemplos

- Freeman, Jones, Pasztor, *Example-Based Super-Resolution* (2002).
- Yang et al., *Image Super-Resolution via Sparse Representation* (2010).
- Romano, Isidoro, Milanfar, **RAISR** (2016) — <https://arxiv.org/abs/1606.01299>

SR con CNN

- Dong et al., **SRCNN** (2014) — <https://arxiv.org/abs/1501.00092>
- Shi et al., **ESPCN** / sub-pixel conv (2016) — <https://arxiv.org/abs/1609.05158>
- Kim et al., **VDSR** (2016) — <https://arxiv.org/abs/1511.04587>
- Lim et al., **EDSR** (2017) — <https://arxiv.org/abs/1707.02921>
- Zhang et al., **RCAN** (2018) — <https://arxiv.org/abs/1807.02758>

SR perceptual / GAN

- Ledig et al., **SRGAN** (2017) — <https://arxiv.org/abs/1609.04802>
- Wang et al., **ESRGAN** (2018) — <https://arxiv.org/abs/1809.00219>
- Wang et al., **Real-ESRGAN** (2021) — <https://arxiv.org/abs/2107.10833> · <https://github.com/xinntao/Real-ESRGAN>
- Zhang et al., **BSRGAN** (2021) — <https://arxiv.org/abs/2103.14006>

Transformers

- Dosovitskiy et al., **ViT** (2020) — <https://arxiv.org/abs/2010.11929>
- Liu et al., **Swin Transformer** (2021) — <https://arxiv.org/abs/2103.14030>
- Liang et al., **SwinIR** (2021) — <https://arxiv.org/abs/2108.10257>
- Conde et al., **Swin2SR** (2022) — <https://arxiv.org/abs/2209.11345> · <https://github.com/mv-lab/swin2sr>

Métricas

- Blau, Michaeli, *The Perception-Distortion Tradeoff* (2018) — <https://arxiv.org/abs/1711.06077>
- Zhang et al., **LPIPS** (2018) — <https://arxiv.org/abs/1801.03924>

Herramientas / modelos preentrenados

- 🤗 Transformers (Swin2SR) — https://huggingface.co/docs/transformers/model_doc/swin2sr
- Modelos **caidas/swin2SR-*** — <https://huggingface.co/caidas>
- Real-ESRGAN (wrapper sencillo) — <https://github.com/ai-forever/Real-ESRGAN>